

Comunicazione Asincrona Tra MicroServizi

Introduzione kafka

Il problema di partenza. Il docente parte dall'architettura **client-server**: i client fanno richieste, i server rispondono. Modello semplice e diretto, ma con un limite grosso — **fatica a scalare**. Di fronte a un aumento enorme di richieste, gestire la comunicazione tra tanti client e tanti server diventa un problema.

La risposta: l'architettura guidata dagli eventi (event-driven). Qui i dati non sono più "richieste", sono **eventi**, generati da fonti diverse e di tipo diverso. Lo schema è:

```
Producer 1 ┘  
Producer 2 ┤──> [ componente centrale ] ──> Consumer A  
Producer 3 ┘ (Kafka) ──> Consumer B
```

I **produttori** generano dati, li mandano a un **componente centrale**, e quel componente li **smista ai vari consumer**. Il vantaggio che il docente sottolinea è duplice: si gestisce una mole di dati molto più grande, e la comunicazione diventa **asincrona** — *il produttore non deve aspettare che il consumer risponda*: invia il messaggio e prosegue con quello che deve fare.

Che cos'è Apache Kafka. Una **piattaforma di streaming open source**, progettata per gestire e analizzare **flussi di dati in tempo reale**. "Piattaforma di streaming" significa un sistema capace di elaborare **dati in movimento**: raccogliarli, memorizzarli ed elaborarli in modo continuo.

Storia. Sviluppato da **LinkedIn intorno al 2010**, proprio perché un social network di quelle dimensioni deve gestire in tempo reale una quantità enorme di dati. (E il docente fa la battuta: il nome viene dallo scrittore, Franz Kafka.) Poi è passato alla Apache Software Foundation.

A cosa serve. Integra sistemi molto diversi tra loro — IoT, CRM, database — e negli ultimi anni si è diffuso soprattutto per far **interagire microservizi**.

I 4 vantaggi che l'hanno fatto emergere (li ripete anche nel riepilogo, quindi sono la parte da ricordare):

1. **High throughput** — gestisce migliaia di messaggi al secondo, performance molto elevate.
2. **Scalabilità** — grazie all'architettura distribuita scala **orizzontalmente**: se i messaggi aumentano, aggiungi server Kafka, in modo semplice.
3. **Persistenza dei dati** — Kafka memorizza i messaggi **sull'hard disk**. Attenzione: non in modo permanente, c'è un **limite di tempo** (in genere una settimana). Ma questo basta per **recuperare dati storici** e farci sopra statistiche e analisi.
4. **Resilienza** — alta disponibilità anche in caso di guasti. Se crolla un server, Kafka ti garantisce che i dati non vengono persi.

Nota metodologica del docente: le architetture client-server **non sono morte**, si usano ancora oggi. L'approccio a eventi conviene **quando c'è una mole enorme di dati** (contesti Big Data).

Broker

Un **Kafka broker** non è altro che **un server Kafka**. Sta **in mezzo** tra due entità:

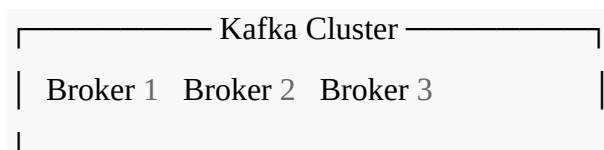
```
Producer —> [ BROKER ] —> Consumer
              (server Kafka)
```

Riceve i messaggi dal **producer**, li memorizza, e li consegna al **consumer** *quando il consumer è pronto*. È esattamente questo "quando è pronto" a rendere la comunicazione asincrona.

Le tre caratteristiche del broker che il docente elenca:

- **Persistenza e ordine cronologico** — il broker riceve i messaggi, li memorizza in modo **persistente** e **in ordine cronologico**. Questo ti dà lo **storico dei messaggi**: se devi recuperare messaggi passati, puoi farlo.
- **Replica dei dati** — i broker permettono di avere **più copie degli stessi dati**. Serve sia a scalare ulteriormente, sia a non perdere niente: se un broker va giù, i dati sono anche altrove.
- **Configurabilità** — possono essere configurati in tantissimi modi (parte dei quali si vedrà nelle lezioni successive).

Kafka Cluster. Un **insieme di broker**, cioè un insieme di server Kafka che lavorano insieme.



È il meccanismo con cui si scala: se le richieste aumentano e servono più prestazioni, **crei un cluster** invece di un broker singolo, e agisci sulle altre configurazioni.

I termini **topic** e **partition** compaiono già nelle slide di questa lezione, ma il docente li rimanda esplicitamente al video successivo. Qui basta fissare: **il rettangolo giallo dello schema = un broker = un server**.

Topic, Partizioni e Offiser in Apache Kafka

Questa è la lezione teorica più densa. Tre concetti, uno dentro l'altro.

Topic

Definizione. Un topic è **un flusso specifico di dati** all'interno di un cluster Kafka. Un cluster può avere **quanti topic vuoi**: un topic dei log, un topic degli acquisti, un topic delle vendite... Ognuno ha **il suo nome**, che è quello che lo identifica.

L'analogia che dà (e i suoi limiti). Puoi immaginare i topic come **le tabelle di un database**, ma **senza i vincoli** delle tabelle:

- su un topic puoi mandare **qualsiasi tipo di messaggio** — testo, file, binario, quello che vuoi;
- ma **non puoi interrogarli**: sui topic **non si fanno query**. Questa è la differenza sostanziale con una tabella.

Data stream. La **sequenza dei messaggi di un topic** si chiama **data stream**. Ed è precisamente per questo che Kafka si chiama *piattaforma di streaming di dati*: perché attraverso i topic crea un flusso.

Le due entità che ci lavorano intorno:

- **Producer** — i componenti che **inviano** dati a un topic;
- **Consumer** — i componenti che **leggono/consumano** i dati da un topic.

⚠ Qui il docente si confonde a voce e dice *"i producer inviano dati ad un database"*: intendeva **a un topic**. Se rileggi la trascrizione, non farti trarre in inganno.

Partizioni

Definizione. Ogni topic può essere **suddiviso in partizioni**, cioè in parti più piccole. Un topic potrebbe essere diviso in 8 partizioni, in 100, quante ne vuoi.

Broker

```
|— Topic 1 — Partizione 0, Partizione 1, Partizione 2 ...  
|— Topic 2 — Partizione 0, Partizione 1 ...  
|— Topic 3 — ...
```

Il punto chiave: quando un producer manda un messaggio a un topic, quel messaggio **finisce in UNA partizione, scelta in modo casuale**. Tu **non sai** in quale finirà.

Il docente si corregge subito e dà l'informazione importante: **un modo per deciderlo c'è** — si usano le **chiavi** (key) — ma lo rimanda a un altro video. Quindi: *nella configurazione standard la partizione è casuale*.

Offset

Definizione. A ogni messaggio che arriva in una partizione viene assegnato **un ID crescente**, e questo ID si chiama **offset**.

Le tre proprietà dell'offset, che sono la parte che va capita bene:

1. **L'offset è per-partizione, non per-topic.** Ogni partizione ha la sua numerazione indipendente. Nell'esempio delle slide: partizione 0 arriva fino a offset 12, partizione 1 fino a offset 8, partizione 2 fino a offset 10.
2. **Stesso offset in partizioni diverse = messaggi diversi.** Il messaggio con offset 1 della partizione 0 **non ha nulla a che vedere** con il messaggio con offset 1 della partizione 1. L'offset da solo non identifica un messaggio: serve la coppia **(partizione, offset)**.
3. **Gli offset non vengono mai riutilizzati.** Se il messaggio con offset 8 della partizione 0 viene eliminato (perché scaduto), il numero 8 **non viene riassegnato** al messaggio successivo. Gli offset sono semplicemente ID che crescono in continuazione.

Immutabilità e retention

Immutabilità. Una volta che un dato è scritto in una partizione, **non può essere né eliminato né aggiornato**. L'unica cosa che si può fare è **continuare a scrivere** in coda alla partizione. Questo è il concetto di **immutabilità dei dati**.

Retention. I dati immutabili vengono conservati per un **periodo di tempo limitato**: il valore predefinito è in genere **una settimana**, ma è **configurabile**. Passata la settimana, spariscono.

Da tenere insieme: *non li puoi modificare, non li puoi cancellare tu, ma dopo un tempo configurato spariscono da soli.*

HelloWord in Kafka

Questa lezione ricapitola l'architettura da un'altra angolazione e poi passa finalmente al codice.

La definizione di comunicazione asincrona

Prima di tutto il docente definisce Kafka: **piattaforma open source di messaggistica distribuita, scritta in Java e Scala**, sviluppata sotto Apache Software Foundation. Serve a permettere la **comunicazione asincrona** tra componenti diversi.

E spiega asincrono partendo dal suo contrario, con due analogie del mondo reale che sono il modo migliore per fissarlo:

	Analogia	Come funziona	Esempio informatico
Sincrona	Il telefono	Devono essere presenti entrambi nello stesso istante . Io chiamo, Tizio deve rispondere adesso. Io aspetto , non vado avanti.	Chiamata a un web service : chiamo e attendo la risposta
Asincrona	L'email	Io mando il messaggio, il destinatario lo legge quando vuole e risponde quando vuole . Non serve la presenza contemporanea.	Kafka

L'architettura, i pezzi in fila

- **Broker** = i server/nodi su cui gira Kafka.
- **Kafka Cluster** = l'insieme dei broker.
- **Topic** = dentro ogni broker, le "categorie"/code in cui i messaggi vengono memorizzati.
- **Producer** = chi invia. E attenzione al dettaglio: il producer **non manda il messaggio "a Kafka" in generale**, gli dice «*ti mando questo messaggio, per il topic A*».
- **Consumer** = chi si **abbona** a un topic. Abbonarsi significa che, quando arriva un messaggio su quel topic, **viene inviato a tutti gli abbonati**.
- **Partizioni** = suddivisioni del topic, fatte per **scalabilità, load balancing e prestazioni**.

Come nascono i topic — due modi:

1. **Esplicitamente** (consigliato dal docente): li configuri direttamente su Kafka, indicando **nome del topic, numero di partizioni e livello di replicazione** (su quanti broker replicarlo). Nell'esempio della slide c'è un *replication factor* = 3: lo stesso topic A con le sue partizioni viene replicato su 3 broker diversi.
2. **A runtime**: mando un messaggio, il topic non esiste, **viene creato in automatico**. Funziona, ma è "teoricamente"; meglio la configurazione esplicita. (*Nel progetto d'esempio però userà proprio questa seconda via — vedi sotto.*)

Consumer group. Quando un consumer si abbona, deve **anche specificare a quale gruppo appartiene** (il *consumer group*). Kafka usa i gruppi per **ottimizzare la distribuzione dei messaggi**. Questo spiega il `group-id` che troverai nel codice in due punti.

⚠️ Altro lapsus della trascrizione: "*garantisce la comunicazione sincrona*" → intendeva **asincrona**.

Passo 1 — Avviare Kafka con Docker Compose

Il docente dice che ci sono **due strade**: seguire la Quick Start del sito ufficiale (scaricare e configurare tutto a mano), oppure — molto più semplice — **usare Docker Compose**. Sceglie la seconda, riagganciandosi alla lezione su Docker.

Il file vero, `docker-compose.yml` dello zip:

version: '3'

services:

zookeeper:

image: confluentinc/cp-zookeeper:latest

container_name: zookeeper

environment:

ZOOKEEPER_SERVER_ID: 1

ZOOKEEPER_CLIENT_PORT: 2181

ZOOKEEPER_TICK_TIME: 2000

ports:

- "22181:2181"

broker:

image: confluentinc/cp-kafka:latest

container_name: broker

ports:

- "9092:9092"

depends_on:

- zookeeper

environment:

KAFKA_BROKER_ID: 1

KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT

KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

```
KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
```

Riga per riga, quello che dice il docente più quello che il file fa davvero:

- **Due container, non uno.** Kafka da solo non basta: serve **Zookeeper**, un componente necessario al funzionamento di Kafka (tiene la configurazione del cluster, i broker, i topic). Gira sulla porta **2181**.
- **depends_on: zookeeper** — esattamente la chiave vista nella lezione su Docker Compose: **prima parte Zookeeper, poi Kafka**.
- **KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181** — il broker raggiunge Zookeeper usando **il nome del servizio come hostname** (di nuovo, il meccanismo delle reti Docker già visto). Nota che qui usa la porta **interna 2181**, non la 22181 mappata all'esterno.
- **ports: "9092:9092"** — il broker è esposto all'esterno sulla **9092**, che è l'indirizzo che l'applicazione Spring userà.
- **KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092** — l'indirizzo che Kafka *comunica ai client* per farsi contattare. È il pezzo che di solito rompe tutto se sbagliato.
- **KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"** — ★ questa è la riga che rende possibile il "topic creato a runtime" di cui sopra. È il motivo per cui nel progetto **nessuno crea my-topic** e funziona lo stesso.
- I tre `..._REPLICATION_FACTOR: 1 / MIN_ISR: 1` sono coerenti con l'avere **un solo broker**: non puoi replicare su 3 nodi se ne hai 1.

Avvio, dalla cartella dove sta il file:

```
docker-compose up
```

(Nota di attualità, non del corso: dalla versione 3.3+ Kafka supporta **KRaft** e Zookeeper è ormai deprecato/rimosso. Se un domani rifai l'esempio con immagini recenti, potresti trovare configurazioni con un solo container. La logica producer/topic/consumer resta identica.)

Passo 2 — Le dipendenze

Il progetto è Spring Boot **3.2.4** con **Java 17**. Dal `pom.xml` reale, le due che contano:

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka-test</artifactId>
  <scope>test</scope>
</dependency>
```

Più `spring-boot-starter-web` (serve il controller REST per far partire il test) e `spring-boot-devtools`.

Passo 3 — La configurazione

application.properties integrale:

```
spring.kafka.bootstrap-servers=localhost:9092
spring.kafka.consumer.group-id=my-consumer-group
spring.kafka.consumer.auto-offset-reset=earliest
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.apache.kafka.common.serialization.StringSerializer
```

Cosa fa ogni riga:

- **bootstrap-servers** — «guarda, il nostro Kafka è su questo indirizzo». È la 9092 esposta dal compose.
- **consumer.group-id** — il **consumer group** di cui parlava nella teoria. Il docente sottolinea che è importante.
- **auto-offset-reset=earliest** — ★ e qui si chiude il cerchio con la **lezione 3**. Dice al consumer: quando non hai un offset salvato per questo gruppo, **parti dal messaggio più vecchio disponibile** (`earliest`) invece che solo dai nuovi (`latest`). È il motivo per cui, riavviando l'app con un `group-id` nuovo, ti ritrovi tutto lo storico.
- **i due serializer** — i messaggi viaggiano su Kafka come **byte**. Qui dichiari che chiave e valore sono **stringhe**, quindi vanno serializzati con `StringSerializer`.

```
package com;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.stereotype.Service;

@Service
public class KafkaProducer {

    private static final String TOPIC = "my-topic";

    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void sendMessage(String message) {
```

```

    System.out.println("message sending");
    kafkaTemplate.send(TOPIC, message);
    System.out.println("message sended");
}
}

```

I punti che evidenzia:

- è un normalissimo **@Service**;
- si inietta con **@Autowired** un **KafkaTemplate<String, String>** — i due generici sono **tipo della chiave** e **tipo del valore**, coerenti con i due **StringSerializer** nel properties;
- **kafkaTemplate.send(TOPIC, message)** è l'unica riga che conta: «*spedisci il messaggio message al topic di nome my-topic*»;
- le due **System.out.println** prima e dopo servono a **vedere in console che l'invio è passato di lì** — nella demo mostra esattamente `message sending` seguito da un bel po' di log di configurazione del producer, poi `message sended`.

Nota: `my-topic` **non è stato creato da nessuna parte**. Viene creato **al volo da Kafka**, grazie a `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"`. Il docente lo dice esplicitamente nel riepilogo: «*in questo caso non esiste, l'ha creato lui in automatico*», ricordando però che **la via consigliata sarebbe configurarlo direttamente su Kafka**.

Passo 5 — Il Consumer

```

package com;

import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Service;

@Service
public class KafkaConsumer {

    @KafkaListener(topics = "my-topic", groupId = "my-consumer-group")
    public void listen(String message) {
        System.out.println("messaggio ricevuto " + message);
    }
}

```

Anche qui: un **@Service** e **un metodo solo**. Il cuore è **@KafkaListener**, che dice «*mettiti in ascolto su questo topic, con questo group id*». Il metodo **non lo chiami tu**: viene invocato da Spring

nel momento in cui arriva un messaggio su my-topic. Il nome listen è arbitrario, chiamalo come vuoi.

Da notare che il groupId compare **due volte**: nel properties (default globale) e nell'annotazione (esplicito per questo listener). Qui coincidono, quindi è ridondante ma chiaro.

Passo 6 — Il controller, solo per far partire tutto

```
package com;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController

public class HelloCtr {

    @Autowired
    KafkaProducer producer;

    //http://localhost:8080/send?msg=ciao

    @GetMapping("/send")
    public String sendMsg(@RequestParam("msg") String message) {
        producer.sendMessage(message);
        return "Message sent to Kafka Topic";
    }
}
```

Il docente precisa che questo controller **non è parte di Kafka**: serve solo a **simulare l'invio di un messaggio** da fuori. Prende il messaggio dalla query string, chiama `producer.sendMessage(...)` e risponde subito con una stringa fissa.

E qui c'è la cosa concettualmente più importante di tutta la lezione: **il controller risponde immediatamente Message sent to Kafka Topic**, senza aspettare che il consumer abbia fatto qualcosa. *Questa è l'asincronia*. Con una chiamata sincrona a un web service avresti aspettato la risposta dell'altro sistema.

Passo 7 — Il test

1. `docker-compose up` → partono zookeeper e broker.

2. Si avvia l'applicazione Spring Boot; in console compaiono i valori di configurazione del **consumer** e il listener risulta avviato con successo.
3. Si chiama nel browser:

<http://localhost:8080/send?msg=ciao>

4. Il browser mostra **Message sent to Kafka Topic**.
5. In console si vede la sequenza completa:

message sending

... (log di configurazione del producer)

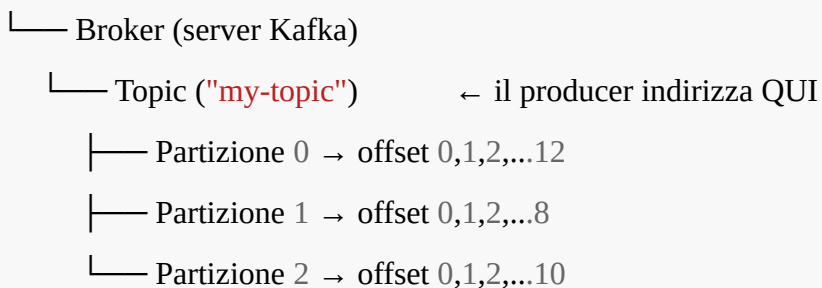
message sended

messaggio ricevuto ciao

Le prime tre righe vengono dal **producer**, l'ultima dal **consumer** — che è un componente completamente separato, che non è stato chiamato da nessuno: si è **svegliato perché è arrivato un messaggio sul topic a cui è abbonato**. In questo esempio producer e consumer stanno nella stessa applicazione per comodità didattica, ma nella realtà sarebbero **due microservizi diversi, su macchine diverse**.

Riepilogo dei concetti in una mappa

Kafka Cluster



Producer —send()—> Topic —push—> Consumer (abbonato, con consumer group)

E la mappa concetto → codice:

Concetto teorico	Dove lo vedi nel progetto
Broker	container broker, porta 9092
Cluster	un solo broker → <code>REPLICATION_FACTOR: 1</code>
Topic	"my-topic", creato in automatico
Producer	<code>KafkaProducer + KafkaTemplate.send()</code>
Consumer + group	<code>@KafkaListener(topics=..., groupId=...)</code>
Offset / retention	<code>auto-offset-reset=earliest</code>
Asincronia	il controller risponde subito, senza attendere il consumer

